



Political Patricians (patricians)

Een romeinse censor is N patriciërs met dubieuze politieke allianties in de gaten aan het houden. Elke patriciër behoort to **exact één** geheime politieke factie, en elke factie heeft een of meer patriciërs. De censor wil bepalen welke patriciërs bij welke facties horen.

Om dat te doen zal de censor meermaals een banket organiseren waar alle N patriciërs worden uitgenodigd. Bij ieder banket kunnen de patriciërs aan verschillende tafels worden geplaatst, en de censor bepaalt de zitplaatsindeling. Een patriciër is *eenzaam* als er geen andere patriciërs van dezelfde politieke factie aan dezelfde tafel zitten.

Als de ceremoniemeester weet de censor na elk banket welke patriciërs eenzaam waren. Jouw opdracht is om de censor te helpen de politieke loyaliteiten van de patriciërs te bepalen zonder te veel banketten te organiseren.

Implementatie

Je moet één .cpp bronbestand inleveren.



In de bijlagen van deze taak vind je een sjabloon `patricians.cpp` met een voorbeeld implementatie.

Je moet de volgende functie implementeren:

C++	<code>vector<int> find_factions(int N)</code>
-----	---

- Integer N staat voor het aantal patriciërs. Patriciërs zijn genummerd van 0 tot en met $N - 1$.
- De functie moet een array P met N integers teruggeven, die staan voor de politieke facties van de patriciërs. $P[i]$ moet gelijk zijn aan $P[j]$ dan en slechts dan als patriciërs i en j tot dezelfde politieke factie behoren. Er zijn geen andere eisen aan de waarden van de integers van P .

Je programma mag de volgende functie aanroepen, die al gedefinieerd is in de grader:

C++	<code>vector<bool> organize_banquet(const vector<int>& Q)</code>
-----	--

- De array Q met N integers die staan voor de zitplaatsindeling: patricier i zal aan tafel $Q[i]$ worden geplaatst. Er zijn geen eisen aan de waarden van de integers van Q : tafels kunnen elke nummering worden gegeven, en de enige relevante informatie is dat wanneer $Q[i] = Q[j]$, dan zullen patriciër i en patriciër j aan dezelfde tafel worden geplaatst.
- De functie zal een array A met N booleans teruggeven. $A[i]$ is waar dan en slechts dan als patriciër i eenzaam is bij het banket.

De grader is **niet adaptief** en elke inzending wordt beoordeeld op dezelfde set testgevallen. Oftewel, de politieke facties van elke patriciër zijn vastgesteld voor elk testgeval en zijn niet afhankelijk van aanroepen naar `organize_banquet` die je programma maakt.

Voorbeeld Grader

In de bijlagen van deze taak vind je een versimpelde versie van de grader die gebruikt wordt tijdens de evaluatie, die je kunt gebruiken om je oplossingen lokaal te testen. De voorbeeld grader leest de

invoer van `stdin`, roept de functie `find_factions` aan en schrijft uitvoer naar `stdout` in het volgende formaat.

De invoer bestaat uit 2 regels, waaronder:

- Regel 1: het integer N .
- Regel 2: N integers die de politieke facties van elke patriciër beschrijven.

De uitvoer bestaat uit één regel, die de score geeft van jouw oplossing voor het testgeval. Als de score 0 is print die een kort bericht naar `stderr` met de reden (onjuiste facties als antwoord gegeven, onjuiste query, of te veel queries).

Randvoorwaarden

- $4 \leq N \leq 2000$.

Je mag `organize_banquet` hooguit 30000 keer aanroepen.

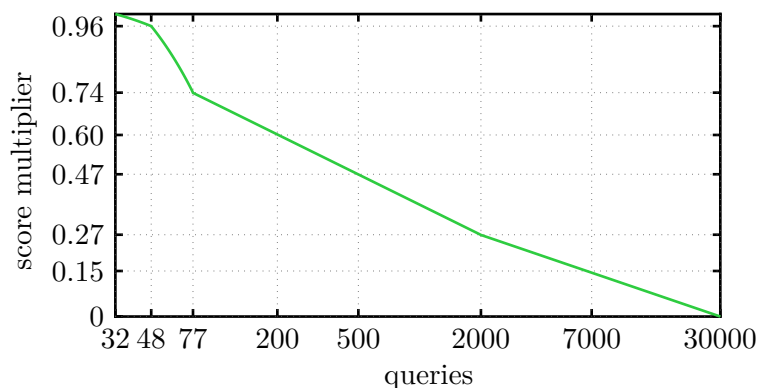
Scoring

Je programma zal getest worden met meerdere testgevallen gegroepeerd in subtaken. De score vermenigvuldiger voor een subtaak is het minimum van de score vermenigvuldigers van elk van de testgevallen.

- **Subtask 0 [0 punten]:** Voorbeeld testgevallen.
- **Subtask 1 [19 punten]:** Er zijn precies twee politieke facties.
- **Subtask 2 [29 punten]:** Elke politieke factie bevat precies twee patriciërs.
- **Subtask 3 [52 punten]:** Geen extra randvoorwaarden.

De score vermenigvuldiger verkregen bij een subtaak is bepaald als volgt.

Aantal queries	Score vermenigvuldiger
≤ 32	1.0
$33 \leq x < 48$	$1.08 - 0.0025x$
48	0.96
$48 < x < 77$	$1.32413 - 0.00758x$
77	0.74
$77 < x < 2000$	$1.366 - 0.1 \log_2(x)$
2000	0.27
$2000 < x \leq 30000$	$1.027 - 0.0691 \log_2(x)$



De score verkregen voor een subtaak is de minimum score vermenigvuldiger verkregen bij de testgevallen daarvan, vermenigvuldigd met de basisscore van de subtaak.

Voorbeelden

Grader	Oplossing
<code>find_factions(5)</code>	
	<code>organize_banquet([0,0,1,1,2])</code>
<code>return [1,1,1,1,1]</code>	
	<code>organize_banquet([0,0,0,1,1])</code>
<code>return [0,1,0,1,1]</code>	
	<code>organize_banquet([0,1,2,1,1])</code>
<code>return [1,0,1,0,1]</code>	
	<code>return [0,1,0,1,2]</code>

Grader	Oplossing
<code>find_factions(4)</code>	
	<code>organize_banquet([1,1,0,0])</code>
<code>return [1,1,1,1]</code>	
	<code>organize_banquet([1,1,1,2])</code>
<code>return [0,1,0,1]</code>	
	<code>organize_banquet([0,1,1,1])</code>
<code>return [1,0,1,0]</code>	
	<code>return [0,1,0,1]</code>

Uitleg

In het eerste voorbeeld zijn patriciërs 0 en 2 in één politieke factie, patriciërs 1 en 3 in een andere factie en patriciër 4 in zijn eigen factie. In het eerste banket zijn alle patriciërs eenzaam. In het tweede banket zijn patriciërs 0 en 2 niet eenzaam omdat ze samen zitten. In het derde banket zijn patriciërs 1 en 3 niet eenzaam omdat ze samen zitten.